

Scholario Ops — Project Document

Oct 6, 2026 · @Adobe-Byline-7

1. Executive summary

Scholario Ops is a global, application-level operations platform that monitors every application and every VPS an organisation runs, across Production (PRD) and Disaster Recovery (DR) environments. It replaces manual checks across many consoles with one live control centre for availability, infrastructure health, incidents and DR readiness, built only from real data.

The platform is application-agnostic. Any number of applications, servers, providers and regions are registered in the dashboard, and each application has its own PRD and DR servers, URLs, tier, RTO/RPO targets, and either a Cloudflare Load Balancer or a DNS failover record.

Problem. Teams running many applications on many VPS check URLs, hosting consoles, Cloudflare and each server separately. Outages are noticed late, capacity problems are found only after they cause downtime, and DR readiness is rarely measured.

Solution. Scholario Ops probes every application endpoint on a schedule (30 seconds by default), reads Cloudflare Load Balancer and DNS health, collects CPU, memory, disk and service telemetry from a lightweight agent on every VPS, raises incidents automatically, alerts on-call teams, and scores DR readiness for each application with 13 checks.

Users. Operations administrators, operators who respond to incidents, and viewers such as management who need status at a glance.

2. Objectives and scope

The project delivers one trusted control centre for PRD and DR health, with every status taken from a live source and missing data shown as UNKNOWN, never as healthy.

Objectives

1. Detect application outages within one minute (30-second HTTPS checks, confirmed failures).
2. Show Cloudflare Load Balancer pool and origin health next to our own probes.
3. Measure DR readiness continuously instead of during a disaster.
4. Raise, track and resolve incidents with a full audit trail.
5. Alert the right people through Teams, Email, Webhook or PagerDuty.

6. Support any number of applications and VPS, configured in the dashboard and stored in PostgreSQL, with nothing hard-coded.

In scope

Area	Included
Monitoring	HTTP/HTTPS, TCP, DNS and TLS certificate probes; check history for availability and SLA
Infrastructure	Agent telemetry (CPU, memory, disk, load, network, services, processes); Hostinger VPS import
Cloudflare	Zones, DNS records, Load Balancer pools, origin health and RTT for every application
Resilience	DR readiness (13 checks), failover pre-flight, approve or reject decision, backups view
Incident management	Auto-create, acknowledge, assign, notes, severity, resolve, timeline
Operations	Maintenance windows, runbooks, deployment and backup reports from CI
Administration	Users and four roles, audit log, notification channels, escalation policies

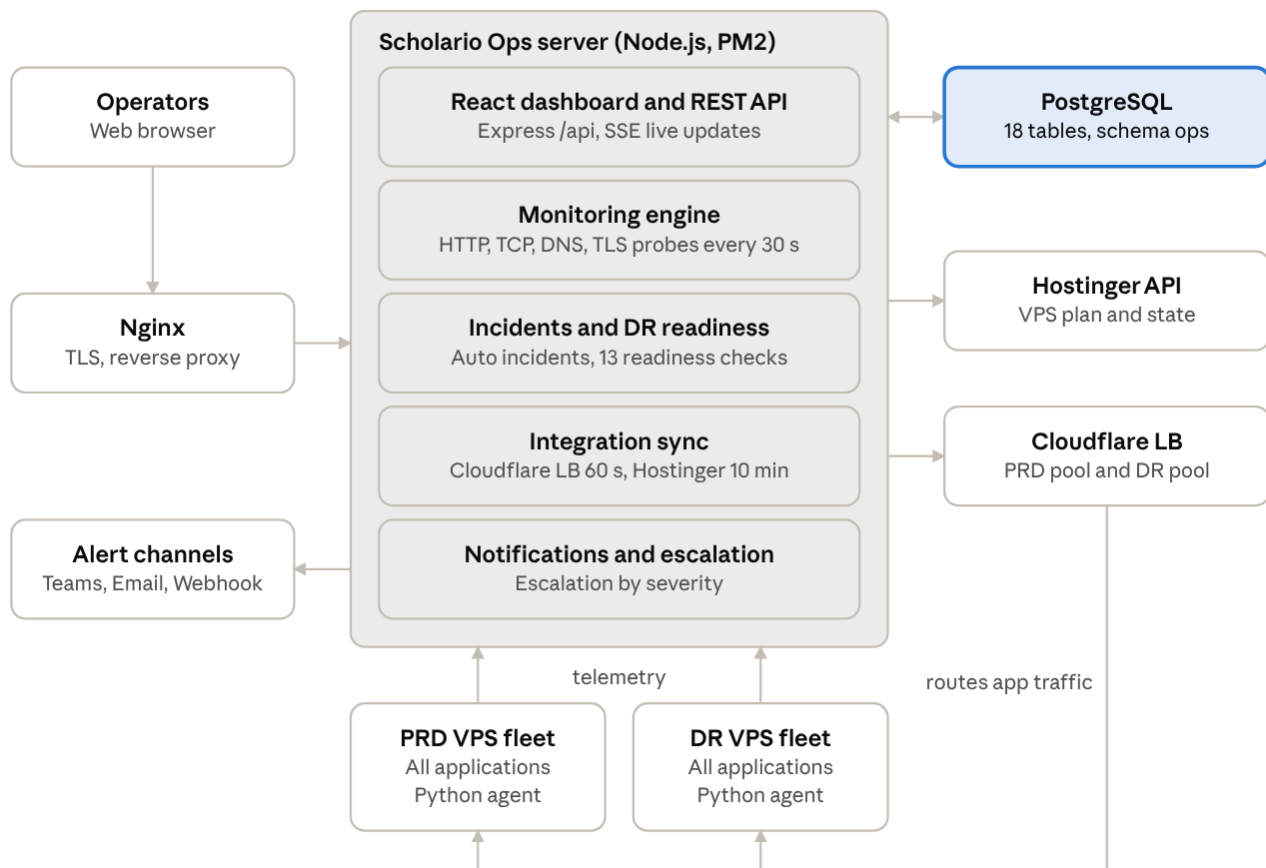
Out of scope

- Automatic failover for Load Balancer applications: the console runs pre-flight checks, shows the manual Cloudflare steps and records the decision. Applications using DNS-record failover can opt in to automatic failover.
- Deploying or patching application code on the VPS.
- Running the backups themselves; the dashboard only receives backup reports.

3. System architecture

One Node.js process on the Ops server runs the dashboard, the API, the monitoring engine and all schedulers, and stores everything in PostgreSQL.

Every live source feeds one server; PostgreSQL is the single source of truth



System architecture · Ops server, data store, VPS pair and integrations

Operators reach the dashboard through Nginx. The engine probes the PRD and DR URLs from outside, agents on both VPS push telemetry over outbound HTTPS, and the server reads Cloudflare and Hostinger through their APIs. Alerts go out to the team's channels.

Data flow for an outage

1. The monitoring engine probes the PRD URL every 30 s and stores each result in `check_results`.
2. After the configured number of consecutive failures, the monitor turns CRITICAL.
3. The engine checks Cloudflare origin health to confirm the outage and opens one de-duplicated incident.
4. Notifications go to the configured channels, escalating by severity until acknowledged.
5. The DR dashboard and failover pre-flight show whether DR is READY; an operator follows the Cloudflare steps and the decision is audited.
6. When checks pass again for the recovery threshold, the incident is resolved automatically.

Deployment topology

Node	Role	Specification
Ops server	Runs Scholario Ops, PostgreSQL and Nginx	Ubuntu VPS, Node.js 20+, PM2
PRD VPS fleet	Serves live traffic for each registered application; runs the telemetry agent	Any Linux VPS, any provider or region; Hostinger plan data imported automatically
DR VPS fleet	Standby copy of each application; runs the telemetry agent	Any Linux VPS, ideally in a different region from PRD
Cloudflare LB or DNS record	Routes each application's traffic to its PRD or DR pool	Configured per application

4. Technology stack

The whole system is TypeScript on Node.js 20, with one process serving both the API and the React dashboard.

Layer	Technology	Purpose
Frontend	React 19, React Router 6, Vite 8	Single-page dashboard with deep-linked views
Styling and UI	Tailwind CSS 4, Lucide icons, Motion	Layout, icons and animation
Backend	Node.js 20+, Express 4, TypeScript	REST API, monitoring engine, schedulers
Database	PostgreSQL (<code>pg</code> driver), schema <code>ops</code>	Single source of truth for configuration and operational data
Realtime	Server-Sent Events (<code>/api/v1/realtime/stream</code>)	Pushes status changes to open dashboards
Agent	Python 3 (standard library only), systemd	Telemetry from each Linux VPS over outbound HTTPS
Email	Nodemailer (SMTP)	Email notification channel
Build	Vite (frontend), esbuild (server bundle)	Produces <code>dist/</code> and <code>dist-server/server.js</code>

Layer	Technology	Purpose
Process manager	PM2 (fork mode, single instance)	Keeps the service running and restarts on failure
Reverse proxy	Nginx	TLS, proxies <code>/api</code> , no buffering for the realtime stream
External APIs	Cloudflare API, Hostinger VPS API	Load Balancer health, DNS, zone data, VPS plan data
Testing	Node test runner via <code>tsx --test</code>	End-to-end tests against throw-away PostgreSQL schemas

5. Functional modules

The application is organised into ten modules, each with its own dashboard view and API group.

Module	What it does	Main views
Authentication and users	Login, token refresh, logout, password change; user management with four roles	Login, Users
Setup and connections	Register servers and applications, PRD/DR URLs, Cloudflare LB pools, DNS failover settings, agent install	Setup
Monitoring engine	Runs HTTP/HTTPS, TCP, DNS, TLS and heartbeat monitors on a 1-second scheduler; confirms failures and recoveries over several consecutive checks	Monitors
Infrastructure telemetry	Python agent on each VPS reports CPU, RAM, disk, load, network, services, top processes and error logs; marked STALE after 60 s of silence	Infrastructure

Module	What it does	Main views
Cloudflare and Hostinger	Reads zones, DNS, LB pools, origin health and RTT (every 60 s); imports VPS plan and state from Hostinger (every 10 min)	Cloudflare, Hostinger, Load Balancer panel
Incident management	Auto-creates de-duplicated incidents on confirmed DOWN or unhealthy origin; acknowledge, assign, notes, severity, auto-recovery	Incidents, Incident timeline
Notifications and escalation	Teams, Email, Webhook and PagerDuty channels; escalation policies per severity; test-send	Communications
DR and failover	13-check DR readiness score; failover pre-flight; manual Cloudflare steps; approve or reject decision written to audit	DR dashboard, LB failover console, Dependency map
Operations	Maintenance windows (pause alerts), runbooks with step checklists, deployment and backup reports from CI	Maintenance, Runbooks, Deployments, Backups
Reporting and audit	Availability and SLA from stored check results, daily and uptime reports, full audit log	Overview, Reports, Audit logs

Monitor result classes. Every check is recorded as UP, DEGRADED, DOWN, TIMEOUT, DNS_ERROR, TLS_ERROR or CONNECTION_ERROR.

DR readiness verdicts. Each check returns PASS, FAIL, WARNING or UNKNOWN. The overall result is READY, DEGRADED, NOT READY or UNKNOWN; any FAIL makes it NOT READY.

Dead-man switch. The server pings an external heartbeat service every 60 s, so someone is alerted if Scholario Ops itself goes down.

6. Data model

All data lives in 18 PostgreSQL tables inside one schema (`ops` by default), created by migration `001_init.sql` at start-up. Administrator-entered configuration is stored in typed columns; volatile live state is kept in a `state` JSONB column.

Table	Holds	Key relationships
<code>users</code>	Email, name, role, active and on-call flags, script password hash, token version	Parent of <code>refresh_tokens</code>
<code>refresh_tokens</code>	Active refresh sessions with expiry	<code>user_id</code> → <code>users</code>
<code>servers</code>	Hostname, IP, PRD/DR role, provider, region, plan CPU/RAM/disk, agent token, Hostinger VM id	Linked from <code>applications</code> and <code>monitors</code>
<code>applications</code>	Name, code, tier, RTO/RPO targets, PRD/DR URLs, Cloudflare zone, DNS record, failover state, health-check settings	<code>prd_server_id</code> , <code>dr_server_id</code> → <code>servers</code>
<code>application_load_balancers</code>	Cloudflare account, LB hostname, PRD pool id, DR pool id	One per application
<code>monitors</code>	Type, target, interval, timeout, thresholds, confirmation counts, expected status, <code>managed_by</code> flag	<code>application_id</code> → <code>applications</code> ; <code>server_id</code> → <code>servers</code>
<code>check_results</code>	Every individual check: time, OK flag, probe status, HTTP code, latency, reason	Indexed by monitor and time
<code>monitor_buckets</code>	Hourly totals for uptime and average latency	Per monitor per hour
<code>server_metrics</code>	1-minute rollups of CPU, RAM, disk, load, network in/out	Per server per minute

Table	Holds	Key relationships
<code>incidents</code>	Title, severity, status, fingerprint, root cause, owner, acknowledgement, timeline (JSONB)	References application and environment
<code>audit_logs</code>	Timestamp, operator, action, category, target, details	Written by every change
<code>channels</code> , <code>escalation_policies</code> , <code>maintenance_windows</code> , <code>runbooks</code> , <code>deployments</code> , <code>backups</code>	Operational records stored as JSONB documents	Referenced by id from monitors and incidents
<code>meta</code>	Key-value system settings	None

Database rules enforce valid data: roles and environments are restricted lists, Cloudflare ids must be 32-character hex, and plan capacity must be positive.

7. API reference

The REST API is served under `/api` and returns JSON. Read endpoints need any signed-in user; the minimum role for changes is shown below.

Group	Endpoints	Minimum role for changes
Health	<code>GET /health</code>	Public
Auth	<code>POST /auth/login</code> , <code>/auth/refresh</code> , <code>/auth/logout</code> , <code>/auth/change-password</code> ; <code>GET /auth/me</code>	Public / signed-in
Agent and CI	<code>POST /v1/agent/ingest</code> ; <code>GET /v1/agent/install/:serverId</code> , <code>/v1/agent/uninstall</code> ; <code>GET/POST /v1/heartbeat/:token</code> ; <code>POST /v1/deployments/report</code> , <code>/v1/backups/report</code>	Per-server or CI token
System	<code>GET /v1/bootstrap</code> , <code>/v1/integrations</code> , <code>/v1/system/summary</code> , <code>/v1/deadman/status</code> , <code>/v1/realtime/stream</code> (SSE)	Viewer

Group	Endpoints	Minimum role for changes
Servers	GET/POST /v1/servers; GET/PATCH/DELETE /v1/servers/:id; /metrics, /agent, /rotate-token	IT administrator
Applications	GET/POST /v1/applications; GET/PATCH/DELETE /v1/applications/:id; /dr-readiness, /availability	IT administrator
Failover	GET /v1/applications/:id/failover/preflight; POST .../failover/decision, POST .../failover	Operator (pre-flight), super admin (decision)
Monitors	GET/POST /v1/monitors; PATCH/DELETE /v1/monitors/:id; /toggle, /probe, /history; POST /v1/monitors/probe-all	IT administrator (config), operator (probe)
Diagnostic tools	POST /v1/tools/http, /tcp, /dns, /ssl	Operator
Incidents	GET/POST /v1/incidents; /acknowledge, /status, /severity, /assign, /notes, /resolve	Operator
Notifications	GET/POST/PATCH/DELETE /v1/channels; POST /v1/channels/:id/test; GET/PUT /v1/escalation-policies	IT administrator
Maintenance and runbooks	/v1/maintenance (create, complete, delete); /v1/runbooks (CRUD, step toggle, reset)	Operator / IT administrator
Deployments and backups	GET /v1/deployments, GET /v1/backups	Viewer
Cloudflare and Hostinger	GET /v1/cloudflare/zones, /accounts, /lb-pools; GET /v1/loadbalancers; GET /v1/hostinger/vms; POST .../sync	Operator (sync), IT administrator (accounts, pools)
Users and audit	GET/POST /v1/users; PATCH /v1/users/:id; DELETE /v1/users/:id; GET /v1/audit	IT administrator; super admin to delete
Reports	GET /v1/reports/summary, /uptime, /daily	Viewer

8. Security

Access is controlled by signed tokens and four nested roles, and every change is written to the audit log.

Roles (each role has all rights of the roles below it)

Role	Level	Can do
Super admin	4	Everything, including failover decisions and deleting users
IT administrator	3	Configure servers, applications, monitors, channels, runbooks, users
Operator	2	Run probes, handle incidents, maintenance windows, syncs, failover pre-flight
Viewer	1	Read-only access to all dashboards and reports

Controls

- **Passwords** are hashed with scrypt (N = 16384) and a random salt; the first admin password must have at least 10 characters with upper case, lower case and a number.
- **Sessions** use HS256-signed access tokens (8 hours by default) and rotating refresh tokens (7 days). A token version on each user lets an admin revoke all sessions.
- **Brute-force protection** allows 10 failed logins per IP and per account, then blocks for 15 minutes.
- **Agent and CI access** uses per-server and deployment tokens compared in constant time; agent reports more than 300 s off server time are rejected.
- **Secrets** (Cloudflare and Hostinger tokens, SMTP password, JWT secret) stay in `.env` on the server. A test confirms the Cloudflare token never reaches the browser.
- **Network**: the app listens on 127.0.0.1:4000 behind Nginx with TLS 1.2/1.3; agents only make outbound HTTPS calls, so no ports are opened on the VPS.
- **HTTP headers**: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, strict referrer policy, and an allow-list for CORS origins.
- **Least privilege at Cloudflare**: Load Balancer monitoring needs read-only permissions; DNS edit is only needed for applications that use DNS-record failover.
- **Audit**: logins, configuration changes, incident actions and failover decisions are recorded with operator, time and details.

9. Configuration and deployment

The service runs as one PM2 process on an Ubuntu VPS behind Nginx; only `ADMIN_EMAIL` and `DATABASE_URL` are needed to start, and every other setting enables an integration.

Key environment variables (`.env`)

Variable	Default	Purpose
<code>PORT</code> , <code>HOST</code>	3000, 0.0.0.0 (production: 4000, 127.0.0.1)	Listen address
<code>PUBLIC_URL</code>	None	HTTPS address used by agents and notification links
<code>DATABASE_URL</code> , <code>DB_SCHEMA</code>	<code>ops</code> schema	PostgreSQL connection and schema
<code>ADMIN_EMAIL</code> , <code>ADMIN_PASSWORD</code> , <code>ADMIN_NAME</code>	Generated password if empty	First super admin
<code>JWT_SECRET</code> , <code>ACCESS_TOKEN_TTL_SEC</code> , <code>REFRESH_TOKEN_TTL_SEC</code>	Generated, 28800, 604800	Session signing and lifetimes
<code>CLOUDFLARE_API_TOKEN</code> , <code>CLOUDFLARE_ACCOUNT_ID</code>	Empty	Cloudflare read access
<code>CLOUDFLARE_SYNC_INTERVAL_SEC</code> , <code>CLOUDFLARE_LB_SYNC_INTERVAL_SEC</code>	300, 60	Zone and LB poll intervals
<code>HOSTINGER_API_TOKEN</code> , <code>HOSTINGER_SYNC_INTERVAL_SEC</code>	Empty, 600	VPS plan import
<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code> , <code>SMTP_FROM</code>	Port 587	Email alerts
<code>DEADMAN_HEARTBEAT_URL</code> , <code>DEADMAN_INTERVAL_SEC</code> , <code>DEADMAN_TOLERANCE_SEC</code>	Empty, 60, 180	External heartbeat for the Ops server itself
<code>TELEMETRY_STALE_THRESHOLD_SEC</code> , <code>METRICS_RETENTION_HOURS</code>	60, 48	Agent staleness and metric retention
<code>CHECK_HISTORY_RETENTION_DAYS</code> , <code>AUDIT_RETENTION_ENTRIES</code>	90, 5000	History retention

Variable	Default	Purpose
<code>DR_LATENCY_THRESHOLD_MS</code>	2000	DR latency warning level
<code>DEPLOY_REPORT_TOKEN</code>	Empty	Lets CI report deployments

Production install

1. Install Node.js 20+, PostgreSQL, Nginx and PM2 on the Ubuntu VPS.
2. Copy `.env.example` to `.env` and set `DATABASE_URL` , `PUBLIC_URL` and `ADMIN_EMAIL` .
3. Run `npm ci && npm run build` to produce `dist/` and `dist-server/server.js` .
4. Run `npm run db:migrate` (migrations also run automatically at start).
5. Run `pm2 start ecosystem.config.js --env production` , then `pm2 save` and `pm2 startup` .
6. Install `nginx/scholario-ops.conf` with a Let's Encrypt certificate; it proxies to port 4000 and does not buffer the realtime stream.
7. Sign in, then in **Setup** add the PRD and DR servers, run the agent install command on each VPS as root, add the application and add notification channels.

Local development: `npm install` then `npm run dev` serves the API and UI at `http://localhost:3000`.

Backup: all state is in PostgreSQL, so back up with `pg_dump --schema=ops` .

10. Testing and quality

The project has 15 end-to-end tests in two suites, each run against its own throw-away PostgreSQL schema so production data is never touched.

Suite	Tests	What is verified
<code>config-flow.test.ts</code>	11	Migrations create the schema; servers and applications are saved to PostgreSQL; bad input is rejected; managed URL monitors follow edits; Cloudflare pools persist; the Cloudflare token never reaches the browser; configuration reloads after restart; deleting a URL or app cleans up monitors and LB mapping

Suite	Tests	What is verified
<code>loadbalancer.test.ts</code>	4	"Load pools" lists pools from the Cloudflare API; pool health and RTT are parsed; an origin failure routes to DR, raises an incident and then recovers; a missing permission shows <code>PERMISSION_REQUIRED</code> instead of a stale "healthy"

Cloudflare responses are replaced with recorded response shapes, so the Load Balancer tests run without a live account.

How to run

- `npm test` runs both suites (needs `DATABASE_URL`).
- `npm run lint` type-checks the frontend and backend.

Live verification. Each newly registered application is verified end to end before go-live: both URLs answer the monitors, both agents report `CONNECTED`, Cloudflare pools and origins are read, and the DR readiness score is reviewed.

Quality rules built into the code

- Missing or stale data is shown as `UNKNOWN` or `STALE`, never as `PASS`.
- Failures and recoveries need several consecutive checks before status changes, which avoids false alarms.
- Inputs are validated on the server and again by database constraints.

11. Risks, limitations and roadmap

The main limitation is that Scholario Ops runs as a single instance, so its own availability depends on one server; the dead-man heartbeat makes that failure visible.

Risk or limitation	Impact	Mitigation
Single PM2 instance (scheduler and realtime state are in-process)	Ops dashboard down if its host fails	External dead-man heartbeat alerts within 3 minutes; PM2 auto-restart
Load Balancer failover is manual	Recovery time depends on an operator	Pre-flight checks and step-by-step Cloudflare instructions; decision recorded in audit

Risk or limitation	Impact	Mitigation
Cloudflare token missing or under-privileged	LB health shows Not configured or Permission required	Clear status in the UI and README permission list
Agent clock drift	Telemetry reports rejected	300 s skew limit with a message to enable NTP
Login rate limit kept in memory	Counters reset on restart	Acceptable for one instance; move to the database if scaled out
PostgreSQL holds all state	Data loss if not backed up	Scheduled <code>pg_dump -- schema=ops backups</code>

Roadmap

1. Onboard every application and VPS in the organisation through the same Setup flow.
2. Optional automatic Load Balancer failover with approval guard rails.
3. Run a second Ops instance with shared state for high availability.
4. Scheduled SLA reports sent by email to management.
5. Wider test coverage for incidents, notifications and the agent.

12. Conclusion

Scholario Ops gives the operations team one live, trustworthy view of PRD and DR health across every application and VPS, built only from real probes, agent telemetry and Cloudflare data. It shortens outage detection to under a minute, measures DR readiness every day, and records every incident and failover decision for review. New applications, servers and providers are added from the dashboard without code changes, so the platform grows with the organisation.

Appendix A. Repository layout

Path	Contents
<code>server/</code>	Backend: <code>server.ts</code> (entry), <code>routes.ts</code> (API), <code>engine.ts</code> (monitor scheduler, incidents, dead-man), <code>probes.ts</code> , <code>readiness.ts</code> , <code>loadbalancer.ts</code> , <code>cloudflare.ts</code> , <code>hostinger.ts</code> , <code>agent.ts</code> , <code>notify.ts</code> , <code>auth.ts</code> , <code>store.ts</code> , <code>db.ts</code>
<code>server/migrations/</code>	<code>001_init.sql</code> PostgreSQL schema
<code>src/</code>	React frontend: <code>App.tsx</code> , <code>components/</code> (views by area), <code>context/</code> , <code>services/</code> , <code>types/</code>
<code>tests/</code>	End-to-end test suites
<code>nginx/</code>	Reverse-proxy configuration
<code>ecosystem.config.js</code>	PM2 production configuration
<code>.env.example</code>	All configuration variables with explanations

Appendix B. Glossary

Term	Meaning
PRD	Production (primary) environment
DR	Disaster Recovery (standby) environment
RTO	Recovery Time Objective: maximum acceptable downtime
RPO	Recovery Point Objective: maximum acceptable data loss
LB	Cloudflare Load Balancer that routes traffic between the PRD and DR pools
Pool / origin	A group of servers in the LB / one server address inside a pool
RTT	Round-trip time measured by Cloudflare to an origin

Term	Meaning
Agent	Python service on each VPS that sends telemetry to Scholario Ops
Dead-man switch	External heartbeat that alerts when Scholario Ops itself stops
SSE	Server-Sent Events, used to push live updates to the browser